

Wireless link layer: error control and wrap-up

Mukulika Maity

Ack: Mythili Vutukuru

Error control

- Error control in the form of link layer retransmissions required to recover from errors on the wireless channel
- Most of 802.11 (WiFi) uses **Stop-and-Wait ARQ** (automatic repeat request) protocol. That is, after sending a frame, the sender waits for a certain period of time for the link layer ACK. If no ACK is received, sender retransmits the frame multiple times. If the maximum retry limit is reached, sender abandons the frame and moves on.
- **Successive retries are not always sent at the original bit rate**. Different implementations of device drivers use different techniques, but usually, bit rate is dropped a notch every few retries, and the last retry of the frame is usually at the lowest bit rate.
- Do not forget that successive retries have longer and longer backoff times too (due to doubling of contention window)

Link layer overheads

- The various waiting times at the MAC layer, coupled with the time to wait for an ACK, lead to a much lower throughput at the link layer than what is offered by the physical layer.
- Typically, when TCP is used, the TCP ACK is also a payload at the link layer, and has its own link layer ACK. Therefore, the overhead of a TCP segment includes the various MAC-level overheads of the data and for the TCP ACK as well.
- Therefore, for a typical bit rate like 54 Mbps, the best case TCP throughput is only about half the bit rate.

802.11g theoretical throughput

- TCP Data $T_{data} \Rightarrow DIFS + Backoff + Data + SIFS + 802.11ack$
- TCP Ack $T_{ack} \Rightarrow DIFS + Backoff + TCP-Ack + SIFS + 802.11ack$
- TCP Throughput $= (2 * L_{data}) / (2 * T_{data} + T_{ack})$
- Theoretical limit 25Mbps

802.11g theoretical throughput

- Data = $(1460 + 20 + 20 + 36) * 8 \text{ bits} = 12288 \text{ bits}$
- 802.11g adds 6 bits at the end for encoding purpose
- Total data bits = $12288 + 6 \text{ tail bits} = 12294$
- 54Mbps $\Rightarrow$ 64QAM, coding rate $\frac{3}{4}$
- 48 data subcarriers
- 1 symbol = $48 * 6 * \frac{3}{4} = 216 \text{ bits}$
- Total symbols = $12294 / 216 = 57 \text{ symbols}$
- Each OFDM symbol $\sim 4 \mu\text{s}$
- So, data-time = $57 * 4 \mu\text{s}$
- 802.11 ACK would require 1 symbol = $4 \mu\text{s}$
- Each frame requires a $20 \mu\text{s}$ preamble for synchronization

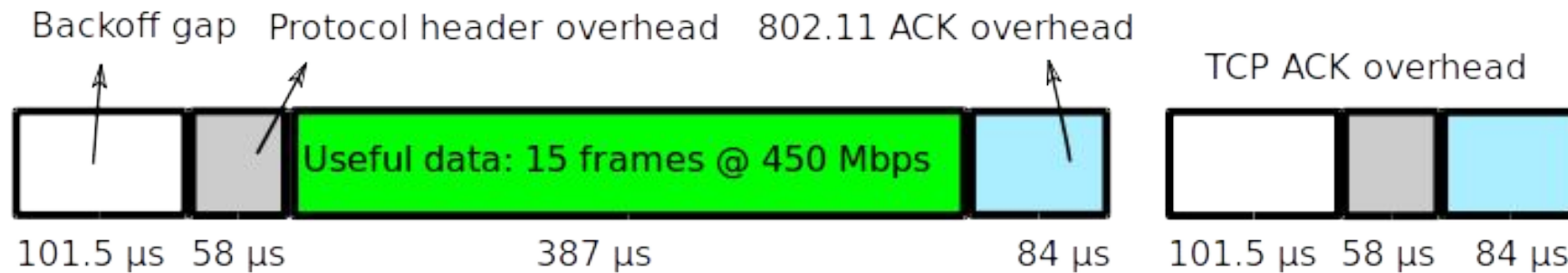
802.11g theoretical throughput

- $T_{data} \Rightarrow DIFS + Backoff + Data + SIFS + 802.11ack$
- $Backoff = \text{Average contention window} \times \text{slot time} = (15/2) \times 9\mu s = 67.5\mu s$
- $T_{Data} = 28 + 67.5 + (20 + 57 \times 4) + 10 + (20 + 4) = 377.5\mu s$
- $T_{ack} \Rightarrow DIFS + Backoff + TCP-Ack + SIFS + 802.11ack$
- $TCP-Ack = (20 + 20 + 36) \times 8 + 6 = 614 \text{ bits}$
- $\text{Total symbols} = 614 / 216 = 3 \text{ symbols}$
- $T_{ack} \Rightarrow 28 + 67.5 + (20 + 3 \times 4) + 10 + (20 + 4) = 161.5\mu s$
- $TCP \text{ Throughput} = (2 \times L_{data}) / (2 \times T_{data} + T_{ack})$
- $TCP \text{ Throughput} = (2 \times 1460) / (2 \times 377.5 + 161.5) = 25.48 \text{ Mbps}$

Frame Aggregation in 802.11n

- The new WiFi standard 802.11n proposes the frame aggregation technique to deal with the problem of MAC Overheads
- Multiple link-layer frames (each with its own CRC) are aggregated and sent back-to-back as one large frame. Backoff for channel contention, waiting for link layer ACK, and all other such overhead-causing actions are performed only once for all the frames.
- A block ACK is used, where a bit map indicates which “subframes” in the aggregated frame have been delivered correctly.
- The block ACK for a certain frame can come after a subsequent frame also, to give the physical layer time to decode the frames. The block ACK covers a window of frames, and the window need not overlap exactly with the sent window and can lag behind a bit.

802.11n MAC/Link Feature: Frame Aggregation



The most crucial feature of 802.11n

Error control in cellular networks

- Cellular data networks use a technique called Hybrid ARQ (HARQ), which combines channel coding and ARQ
- Two variants of HARQ
- **Incremental redundancy**: puncture (remove) some coded parity bits from the original message in the first transmission. Send more parity bits as and when needed in subsequent retransmissions.
 - **Type I**: repeatedly retransmit and discard the previously received data
 - **Type II**: retransmission sequence contains mostly parity bits, than the original sequence, redundant bits are then soft combined with the previously received data
 - **Type III**: is similar to Type II except each packet is self-decodable, more overhead compared to Type II HARQ.

Error control in cellular networks

- **Chase combining**: resend the original coded bits in the retransmission. However, combine both the original transmission and the retransmission to jointly decode the message
- HARQ techniques result in better performance, but are harder to implement

Wireless link-layer wrap-up

- Revise all the functions of the link-layer
 - Medium access
 - Link adaptation
 - Error control
- Walk through an example of a client-AP communication in WiFi, and a 3G/4G data communication in cellular networks from the perspective of the link layer